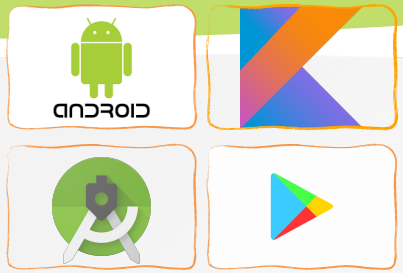
Software Development for Mobile Devices

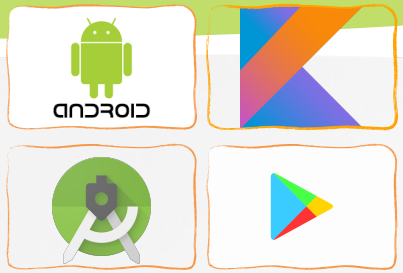
Nguyen Le Hoang Dung
nlhdung@fit.hcmus.edu.vn

Content



- **XML**
- JSON

XML and JSON Data



» It help us securely exchange data on the web
  – Size: small data => less transmission time.
  – Transparency:
    • human-readable quality that allows developers to better understand the nature of their data
    • portable cross-platform solutions more rapidly.

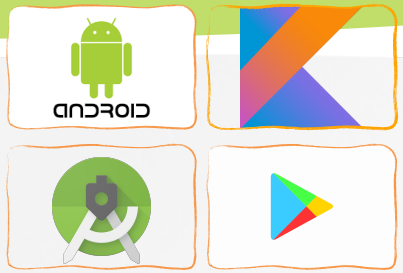
XML Data

» Extensible Markup Language

- **A markup language** was designed to store and share data on the internet
- **XML** was designed to be both human- and machine-readable.
- It's similar to HTML, but without predefined tags to use

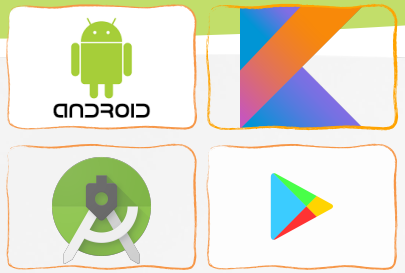
```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>Android course</to>
  <from>Dung</from>
  <body>Don't forget to do your homeowrk!</body>
</note>
```

XML Data
Why should we use XML



- The main role of XML is to facilitate a transparent exchange of data over the Internet.
  - Example: RSS , Atom, SOAP, XHTML, Xquery, Xpath…
- Several document management productivity tools default to XML format for internal data storage.
  - Example: Microsoft Office, AndroidManifest.xml, GoogleService-Info, iOS Info.plist file…
- Android OS relies heavily on XML to save its various resources such as layouts, string-sets, manifest, etc.

Processing XML Data



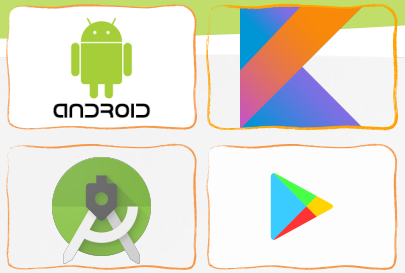
» A SAX (Simple API for XML) XmlPullParser:

- Process a document as it is being read, which avoids the need to wait for all of it to be stored before taking action

» W3C-DOM Document Builder:

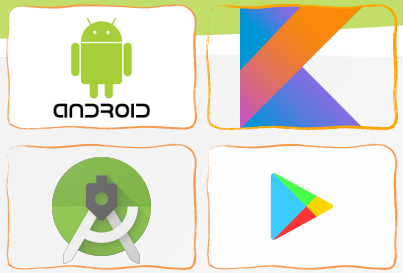
- Document builder object stores the XML document producing an equivalent tree-like representation before taking action.
- Nodes in that tree are treated as familiar Java ArrayLists

Processing XML Data with SAX



» A SAX ***XmlPullParser*** will traverse the document using the next() method to detect the following events
  – XML Text nodes
  – XML Element Starts and Ends
  – XML Processing Instructions
  – XML Comments
» When a tag is recognized, we will use:
  – getName() to get the tag's name.
  – getText() to get text data
  – getAttributeCount(); getAttributeName(); getAttributeValue()

Processing XML Data with SAX



-Element start, named DocElement;

  -attribute param equal to “value" (sorted by parameter’s name)

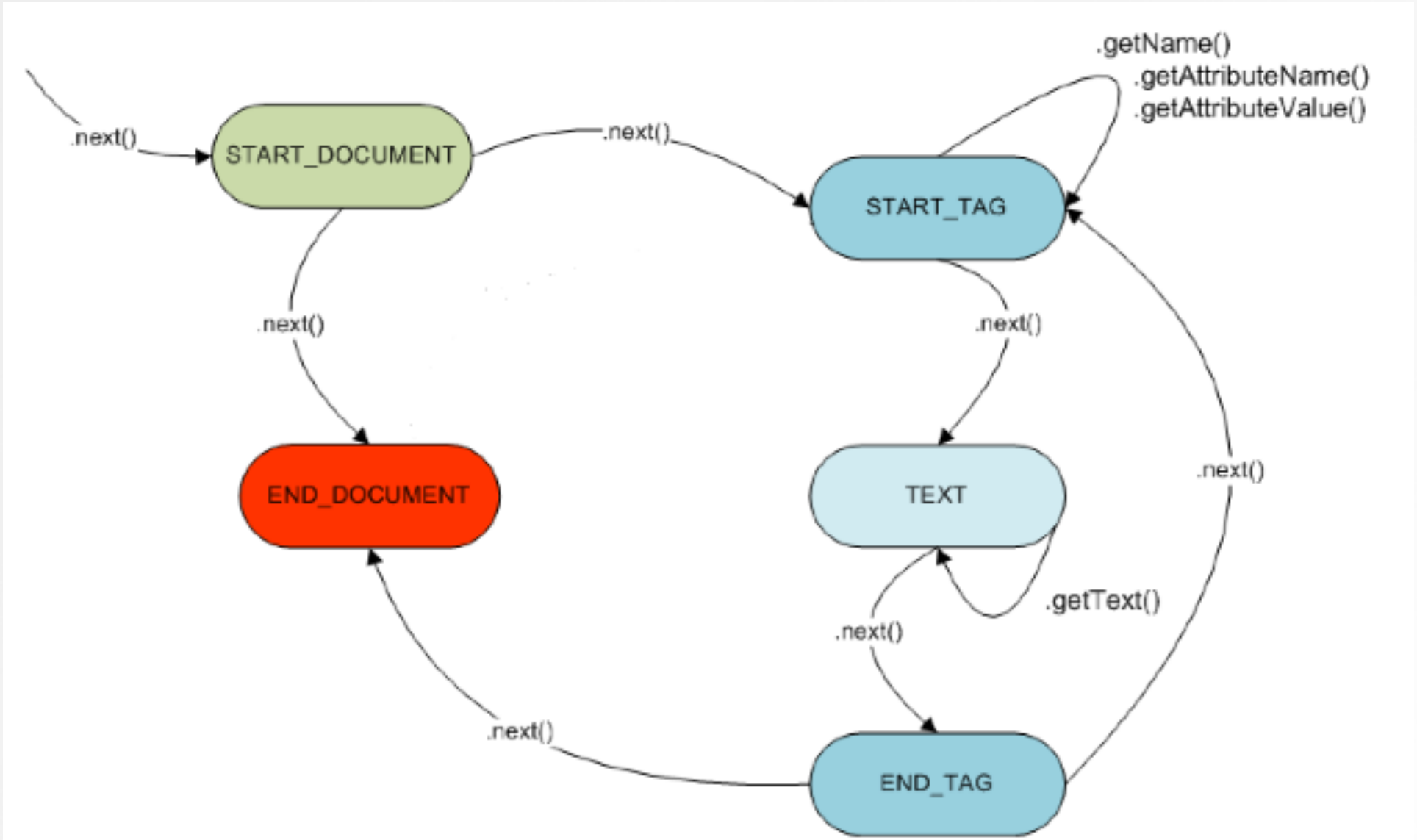
-Element start, named FirstElement

-Text node, with data equal to "¶ Some Text"

-Element end, named FirstElement

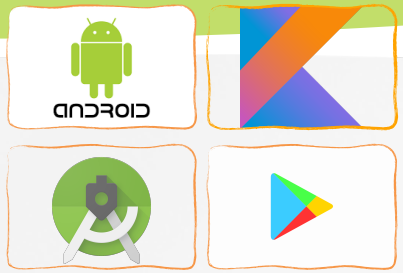
-Processing Instruction event, with the target some_pi and data some_attr="some_value" (the content after the target is just text)

```
<?xml version="1.0" encoding="UTF-8"?>
 <DocElement param="value">
     <FirstElement>
         &#xb6; Some Text
     </FirstElement>
     <?some_pi some_attr="some_value"?>
     <SecondElement param2="something">
         Pre-Text <Inline>Inlined text</Inline> Post-text.
     </SecondElement>
</DocElement>
```



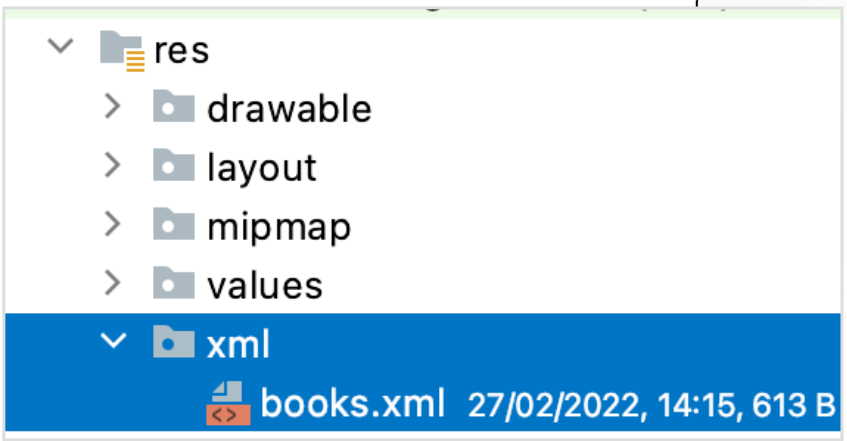
Victor Mats, Android-Chapter14-Reading-XML-Data, Cleveland State University

Processing XML Data with SAX Example 1

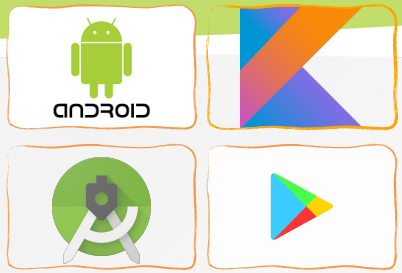


-Add ***books.xml*** into ***res/xml*** folder

```xml
<?xml version="1.0" encoding="utf-8"?>
<bookstore>
    <book category="COOKING">
        <title lang="en">Everyday Italian</title>
        <author>Giada De Laurentiis</author>
        <year>2005</year>
        <price>30.00</price>
    </book>
    <book category="CHILDREN">
        <title lang="en">Harry Potter</title>
        <author>J K. Rowling</author>
        <year>2005</year>
        <price>29.99</price>
    </book>
    <book category="WEB">
        <title lang="en">Learning XML</title>
        <author>Erik T. Ray</author>
        <year>2003</year>
        <price>39.95</price>
    </book>
</bookstore>
```



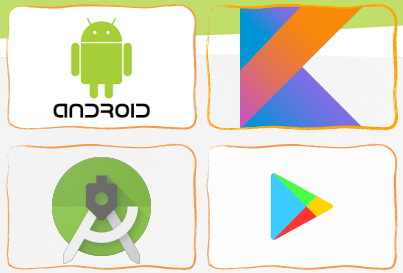
Processing XML Data with SAX Example 1



» Book class

```
//Create Book class with title, author, price, year
class Book(var title: String, var author: String, var price: Double,
     var year: Int, var category: String) {

    //Constructors with parameters
    constructor(title: String, author: String, price: Double, category: String) :
this(title, author, price, 0, category)
    constructor() : this("", "", 0.0, 0, "")

    fun setProperty(name: String, value: String){
        when (name) {
            "title" -> title = value
            "author" -> author = value
            "price" -> price = value.toDouble()
            "year" -> year = value.toInt()
            "category" -> category = value
        }
    }
}
```

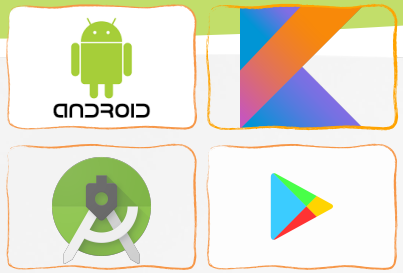
Processing XML Data with SAX Example 1



» Book class

```
//Class BookList class with list of Book
class BookList {
    var list: ArrayList<Book> = ArrayList()
    //Add a book to list
    fun addBook(book: Book) {
        list.add(book)
    }

    //Get a book from list
    fun getBook(index: Int): Book {
        return list.get(index)
    }
    //Get size of list
    fun getSize(): Int {
        return list.size
    }
}
```

Processing XML Data with SAX Example 1

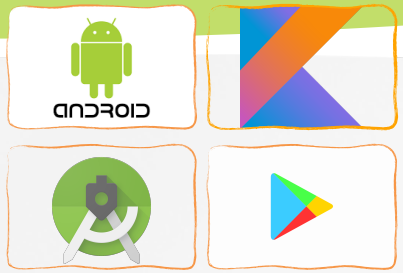


» Processing function

```
fun getXMLContent() : BookList? {
    val parser = resources.getXml(R.xml.books)
    var nodeName = ""
    try {
        var bookList = BookList()
        lateinit var book : Book
        var event = -1
        while (event != XmlPullParser.END_DOCUMENT) {

            //Processing events
            //Create book and insert into bookList


        }

        return bookList
    }
    catch (e: Exception) {
        Log.i("hdlog", e.message.toString())
        return null
    }
}
```

```
//Get next event
event = parser.next()
//Check each events
if (event == XmlPullParser.START_DOCUMENT){
    Log.i("hdlog", "START_DOCUMENT")
}
if(event == XmlPullParser.END_DOCUMENT) {
    Log.i("hdlog", "END_DOCUMENT")
}
if (event == XmlPullParser.START_TAG){
    Log.i("hdlog", "START_TAG")
    if (parser.name == "book") {
        book = Book()
        val att = parser.getAttributeValue(0)
        Log.i("hdlog", att)
        book.setProperty(parser.getAttributeName(0), att)
    }

    if (parser.name == "title" || parser.name == "author" ||
        parser.name == "year" || parser.name == "price") {
        nodeName = parser.name
    }
}
if(event == XmlPullParser.END_TAG) {
    Log.i("hdlog", "END_TAG")
    if (parser.name == "book") {
        bookList.addBook(book)
    }
}
if(event == XmlPullParser.TEXT) {
    book.setProperty(nodeName, parser.text)
}
```

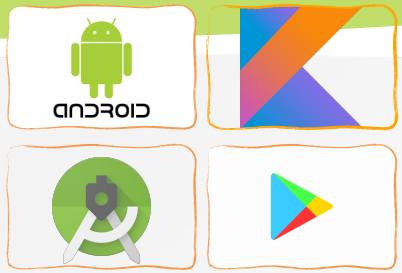
Processing XML Data with SAX Example 1



» Print out bookList

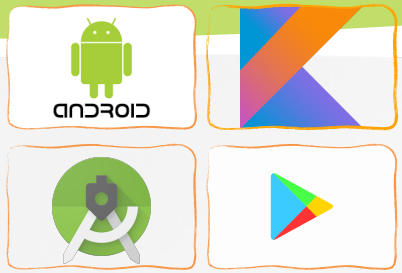
```
fun printBookList(bookList: BookList) {
    //Add each book information into stringBuilder
    val stringBuilder = StringBuilder()
    for (i in 0 until bookList.getSize()) {
        val book = bookList.getBook(i)
        stringBuilder.append("Title: " + book.title + "\n")
        stringBuilder.append("Author: " + book.author + "\n")
        stringBuilder.append("Year: " + book.year + "\n")
        stringBuilder.append("Price: " + book.price + "\n")
        stringBuilder.append("Category: " + book.category + "\n")
        stringBuilder.append("\n")
    }
    //Print all books
    Log.i("hdlog", stringBuilder.toString())
}
```

Processing XML Data with SAX Example 1



- Code comments
- The XML file is in the /res/xml folder.
- The XmlPullParser class using the supplied file resource.
- The while-loop implements the process of stepping through the SAX's parser state diagram.
  - Each call to .next() provides a new event.
  - The if-then clause decides what event is in progress and from there the process continues looking for text, attributes, or end event.
  - When a START_TAG event is detected the parser checks for possible inner attributes, and book's properties.
  - When a END_TAG event is detected , we add book into book list.
  - When a TEXT event is detected, we change book's properties.
  - getAttributeName(), getAttributeValue() extracts attributes (if possible).

Processing XML Data with W3C DOM



» W3C DOM Parser's Strategy:

– <Elements> from the input XML file become nodes in an internal tree. The node labeled <Document> acts as the root of the tree.

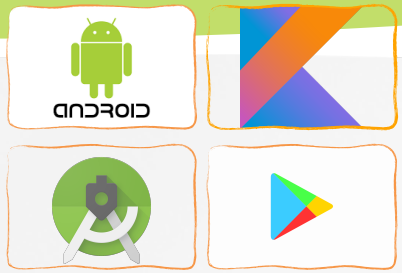
» Steps

– For each selected XML element we get the NodeList collection using ***document.getElementsByTagName(...)***

– Explore an individual node from a NodeList using the methods:

- getElementsByTagName(tagName); list.item(i)
- node.nodeName; node.nodeValue; node.textContent
- node.getFirstChild()
- node.getAttributes()
- ...

Processing XML Data with W3C DOM Example 2

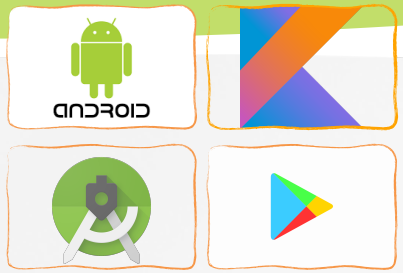


» Add ***books.xml*** file into ***assets*** folder

» Get value for specific node with tag

```
fun getElementValue(tagName:String, node: Node) : String? {
    val child1Nodes = node.childNodes
    for (i2 in 0 until child1Nodes.getLength()) {
        val child1Node = child1Nodes.item(i2)
        if (child1Node.nodeName == tagName) {
            return child1Node.textContent
        }
    }
    return null
}
```

Processing XML Data with W3C DOM Example 2



» Parsing all book nodes

```
fun getXMLWithDOM() : BookList? {
    var bookList : BookList? = null
    try {
        val inputStream = assets.open("books.xml")
        val docBuilder = DocumentBuilderFactory.newInstance().newDocumentBuilder()
        val doc = docBuilder.parse(inputStream)

        val nList = doc.getElementsByTagName("book")
        if (nList.length != 0) { bookList = BookList()}
        for (i in 0 until nList.getLength()) {
            val bookNode = nList.item(i)
            val book = Book()
            book.category = bookNode.attributes.item(0).nodeValue

            book.title = getElementValue("title", bookNode) ?: ""
            book.author = getElementValue("author", bookNode) ?: ""
            book.price = getElementValue("price", bookNode)?.toDouble() ?: 0.0
            book.year =  getElementValue("year", bookNode)?.toInt() ?: 0

            bookList?.addBook(book)
        }
        return bookList
    }
    catch (e: Exception) {
        Log.i("hdlog", e.message.toString())
        return null
    }
}
```